Go for DevOps engineers

Write small, fast, reliable Go services and tools: types, errors, interfaces, goroutines, context, HTTP, testing, containers and debugging.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Why Go for operations, and your first program (12 min)
2. Types, structs, slices and maps (13 min)
3. Functions, errors and defer (13 min)
4. Interfaces and packages (13 min)
5. Goroutines, channels and sync (14 min)
6. Context, timeouts and graceful shutdown (13 min)
7. HTTP services, clients and JSON (13 min)
8. Testing, benchmarks and tooling (12 min)
9. Building CLI tools and small container images (13 min)
10. Debugging Go programs: panics, leaks, races and profiles (14 min)
11. Configuration, logging and metrics in Go services (12 min)
12. Writing Kubernetes clients and controllers in Go (13 min)
Cheat sheet

Lesson 1. Why Go for operations, and your first program

12 minutes. See why so much infrastructure software is written in Go, set up a module, build a single static binary and use the standard tools.

Docker, Kubernetes, Terraform, Prometheus and many other tools you use every day are written in Go. When you read their code, write an operator, or build a small internal tool that must run on any server without installing a runtime, Go is the natural choice. It is a small language with fast builds, built-in concurrency and one standard way to format, test and build code, which makes it easy to pick up for someone who already scripts in Bash or Python.

What you will learn

- Why Go suits operations tools
- Create a module and run, build and cross-compile a program
- Use gofmt, go vet and go mod as part of the normal workflow
- Read a small Go program: packages, imports, main

Why operations teams like Go

Go compiles to a **single static binary** with no interpreter or shared libraries to install, so you can copy one file to a server or put it into an almost empty container image. Compilation is fast, the standard library already includes an HTTP server and client, JSON, testing, and crypto, and goroutines make it practical to handle many connections or checks at once. The language is deliberately small, so code written by different people looks alike. The price is verbosity (explicit error handling) and a less expressive type system than some languages. For glue scripts of ten lines, Bash or Python is still faster to write; Go pays off for tools that others will run, long-running services and anything that must be fast and dependable.

A module and a first program

A Go project is a **module**, described by a `go.mod` file. Code lives in packages, and an executable has a `main` package with a `main` function.

```
mkdir diskcheck && cd diskcheck
go mod init example.com/ops/diskcheck
```

```
package main

import (
    "fmt"
    "os"
)

func main() {
    host, err := os.Hostname()
    if err != nil {
        fmt.Fprintln(os.Stderr, "cannot read hostname:", err)
        os.Exit(1)
    }
    fmt.Printf("checking %s\n", host)
}
```

```
go run .                # compile and run
go build -o diskcheck .  # produce a binary
./diskcheck
```

Notice the pattern: a function returns a value and an `error`, and the caller checks it right away. You will see it everywhere in Go.

MARGA by EknathaLabs marga.eknathalabs.com

Build for any platform

Cross-compiling needs two environment variables and no extra tools:

```
GOOS=linux GOARCH=amd64 go build -o diskcheck-linux-amd64 .
GOOS=linux GOARCH=arm64 go build -o diskcheck-linux-arm64 .
GOOS=darwin GOARCH=arm64 go build -o diskcheck-mac .
file diskcheck-linux-amd64
```

From a laptop you can produce binaries for the servers you manage. Set `CGO_ENABLED=0` when you want a fully static binary that does not depend on the C library, for example for a minimal container image. Check the result with `ldd`, which should say "not a dynamic executable".

The standard toolbox

Go ships with its tools, so teams rarely argue about style or setup:

- `gofmt -l .` lists files that are not formatted; `gofmt -w .` fixes them. Most editors do it on save.
- `go vet ./...` finds suspicious code, such as a wrong format string.
- `go test ./...` runs tests in every package.
- `go mod tidy` adds missing and removes unused dependencies in `go.mod`.
- `go doc fmt.Printf` shows documentation in the terminal.

Dependencies are fetched by version from module proxies and recorded with checksums in `go.sum`, so builds are reproducible. Commit both `go.mod` and `go.sum`. You can practise small exercises in the browser on the Go playground page of this site before installing anything.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Why Go for operations, and your first program fits together

Part	What it does	What goes wrong
Why operations teams like Go	Go compiles to a single static binary with no interpreter or shared libraries to install, so you can copy one file to a server or put it into an almost empty container...	Building on a laptop with CGO enabled and then running on a different C library.
A module and a first program	A Go project is a module, described by a go.mod file.	Ignoring the error return value.
Build for any platform	Cross-compiling needs two environment variables and no extra tools.	Check the Common mistakes list below this diagram for what usually goes wrong here.
The standard toolbox	Go ships with its tools, so teams rarely argue about style or setup.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Why operations teams like Go to A module and a first program / A module and a first program to Build for any platform / Build for any platform to The standard toolbox

Incident: The tool that needed Python 3.11

Incident: *An internal inventory script worked on laptops and failed on half the servers.*

```
$ ./inventory.py
Traceback (most recent call last):
  File "./inventory.py", line 3, in <module>
    import tomllib
ModuleNotFoundError: No module named 'tomllib'
$ python3 --version
Python 3.8.10
```

The script needed a newer interpreter than the older servers had. The team rewrote the tool in Go, built one binary per architecture with GOOS and GOARCH, and shipped it through the artifact store. Nothing

needs to be installed on the servers, and the same binary is used in CI and in the container image.

Common mistakes

- Forgetting to run go mod init so imports fail
- Ignoring the error return value
- Building on a laptop with CGO enabled and then running on a different C library
- Not committing go.sum

Check yourself

What does a Go build produce for a normal command-line tool?

Answer: A single compiled binary

Which file describes a Go module and its dependencies?

Answer: go.mod

How do you build a Linux arm64 binary on a Mac?

Answer: Set GOOS=linux and GOARCH=arm64 and run go build

Which command lists files that are not formatted?

Answer: gofmt -l .

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Types, structs, slices and maps

13 minutes. Use the core data types: zero values, slices, maps and structs with methods, and avoid the common slice and map surprises.

Most operations tools read data, shape it and print or send it: a list of hosts, a map of settings, a record for each disk. Go gives you a handful of simple building blocks for that. They have a few behaviours that surprise people coming from Python, such as zero values, slices that share memory, and maps that are not safe to use from several goroutines at once. Learn those and you avoid most early bugs.

What you will learn

- Variables, constants and zero values
- Slices: length, capacity, append and copying
- Maps: lookup with the comma-ok form
- Structs and methods, including pointer receivers

Variables and zero values

Declare with `var` or the short form `:=` inside functions. Every type has a **zero value** (0, "", false, nil), so a variable is never uninitialized.

```
var count int            // 0
var name string          // ""
threshold := 0.85        // float64, inferred
const maxRetries = 3
```

Types are strict: an `int` and an `int64` cannot be mixed without conversion (`int64(count)`). Strings are immutable sequences of bytes, usually UTF-8, and `len(s)` counts bytes, not characters. Use `strings` and `strconv` from the standard library for most text work, for example `strconv.Atoi("42")` which returns a number and an error.

Slices

A slice is a window onto an array, with a length and a capacity.

```
hosts := []string{"web1", "web2"}
hosts = append(hosts, "web3")     // append may allocate a new array
for i, h := range hosts {
    fmt.Println(i, h)
}
firstTwo := hosts[:2]             // shares memory with hosts
safe := append([]string(nil), hosts...) // an independent copy
```

Two gotchas. First, a sub-slice shares the underlying array, so changing `firstTwo[0]` changes `hosts[0]`; copy when you need independence. Second, always assign the result of `append`, because the slice header you passed may be out of date. A nil slice is valid and behaves like an empty one in `len`, `range` and `append`.

Maps

A map is a hash table with unordered keys.

```
ports := map[string]int{"http": 80, "https": 443}
ports["ssh"] = 22

if p, ok := ports["smtp"]; ok {
    fmt.Println("smtp on", p)
} else {
    fmt.Println("smtp not configured")
}
delete(ports, "ssh")
```

Reading a missing key returns the zero value, so use the **comma-ok** form when "missing" and "zero" mean different things. Writing to a nil map panics, so create one with `make(map[string]int)` or a literal. Iteration order is random on purpose; sort the keys (package `sort` or `slices`) when you need stable output.

MARGA by EknathaLabs marga.eknathalabs.com

A plain map is not safe for concurrent writes, which matters in the lesson on goroutines.

Structs and methods

A struct groups fields; methods attach behaviour.

```
type Disk struct {
    Mount   string
    UsedPct float64
}

func (d Disk) Full() bool { return d.UsedPct >= 90 }

func (d *Disk) Grow(p float64) { d.UsedPct += p }

d := Disk{Mount: "/var", UsedPct: 87}
d.Grow(5)
fmt.Println(d.Full())     // true
```

A **value receiver** works on a copy; a **pointer receiver** can change the original and avoids copying large structs. Mixing both on one type is confusing, so most types use pointer receivers if any method needs to modify state. Exported names start with a capital letter (`Mount`); lowercase names are private to the package. Struct tags such as ` json:"mount" ` control how encoding libraries name fields.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Types, structs, slices and maps fits together

Part	What it does	What goes wrong
Variables and zero values	Declare with var or the short form := inside functions.	Every type has a zero value (0, "", false, nil), so a variable is never uninitialized.
Slices	A slice is a window onto an array, with a length and a capacity.	Forgetting to assign the result of append.
Maps	A map is a hash table with unordered keys.	Relying on map iteration order.
Structs and methods	A struct groups fields; methods attach behaviour.	Assuming a sub-slice is a copy.

Flow: Variables and zero values to Slices / Slices to Maps / Maps to Structs and methods

Incident: The map that panicked at startup

Incident: *A new service crashed on its first request with a panic.*

```
panic: assignment to entry in nil map

goroutine 18 [running]:
main.(*Registry).Add(...)
    /src/registry.go:22
main.handle({0xc0001a2000, 0x1c})
```

The Registry struct had a map field that was never created, because a struct's zero value leaves maps nil. Reading from a nil map is fine, so unit tests passed; the first write panicked. The fix was a constructor function that calls make, and a test that adds an entry. The team also added a lint rule for exported structs with map fields.

Common mistakes

- Writing to a nil map
- Assuming a sub-slice is a copy
- Forgetting to assign the result of append
- Relying on map iteration order

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What is the zero value of a string in Go?

Answer: The empty string

Why use the comma-ok form with a map?

Answer: To tell a missing key from a stored zero value

What happens when you write to a nil map?

Answer: The program panics

When do you need a pointer receiver on a method?

Answer: When the method must modify the value or the struct is large

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Functions, errors and defer

13 minutes. Handle errors the Go way: return them, wrap them with context, inspect them, and use defer for cleanup.

Go has no exceptions for ordinary failures. A function that can fail returns an error as its last value, and the caller decides what to do. This feels repetitive for a day and then becomes a strength: every failure path is visible in the code, which is exactly what you want in software that runs production. This lesson covers the habits that make that style readable: wrapping, inspecting, and cleaning up with defer.

What you will learn

- Functions with multiple return values
- Return, wrap and inspect errors with %w, errors.Is and errors.As
- Use defer for cleanup and understand its order
- When panic and recover are appropriate

Errors are values

```
func readConfig(path string) (Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return Config{}, fmt.Errorf("read config %s: %w", path, err)
    }
    var cfg Config
    if err := json.Unmarshal(data, &cfg); err != nil {
        return Config{}, fmt.Errorf("parse config %s: %w", path, err)
    }
    return cfg, nil
}
```

Check `err != nil` right after the call and return early. Add context with `fmt.Errorf` so the final message tells a story: `read config /etc/app.json: open /etc/app.json: permission denied`. The verb `%w` wraps the original error so it can still be inspected. Never ignore an error with _ unless you can explain why it cannot matter.

Inspect errors

```
_, err := os.Open("/etc/app.json")
if errors.Is(err, os.ErrNotExist) {
    // use defaults
} else if err != nil {
    return err
}

var pe *fs.PathError
if errors.As(err, &pe) {
    fmt.Println("failed path:", pe.Path)
}
```

`errors.Is` compares an error (and everything it wraps) with a known value. `errors.As` finds an error of a specific type in the chain. You can define your own sentinel error with `var ErrNoQuota = errors.New("no quota")` or your own type with an `Error() string` method. Do not compare error strings; they change.

Defer

`defer` schedules a call to run when the surrounding function returns, whatever the path.

```
func copyFile(src, dst string) error {
    in, err := os.Open(src)
    if err != nil {
        return err
    }
    defer in.Close()

    out, err := os.Create(dst)
    if err != nil {
        return err
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    }
    defer out.Close()

    _, err = io.Copy(out, in)
    return err
}
```

Deferred calls run in last-in, first-out order. Arguments are evaluated when the defer statement runs, not when the call happens. Avoid defer inside a long loop, because the calls pile up until the function ends; move the body into a function instead. Deferred calls are how you release locks, close files, and stop timers without leaking them.

Panic and recover

`panic` stops normal flow and unwinds the stack. It is for programmer errors that should never happen (an impossible state, an index out of range) and not for "file not found". A web server framework usually calls `recover` in middleware so that one bad request does not kill the process, and logs the stack trace. In your own code, prefer returning errors. At the top of `main`, print a clear message to standard error and exit with a non-zero code so scripts and orchestrators see the failure:

```
if err := run(); err != nil {
    fmt.Fprintln(os.Stderr, "error:", err)
    os.Exit(1)
}
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Functions, errors and defer fits together

Part	What it does	What goes wrong
Errors are values	Check err != nil right after the call and return early.	Ignoring errors with _.
Inspect errors	errors.Is compares an error (and everything it wraps) with a known value.	Returning errors without context.
Defer	defer schedules a call to run when the surrounding function returns, whatever the path.	Deferring in a loop and holding files open.
Panic and recover	panic stops normal flow and unwinds the stack.	Comparing error text.

Flow: Errors are values to Inspect errors / Inspect errors to Defer / Defer to Panic and recover

Incident: The error that said only "failed"

Incident: *A nightly job logged one word and no one could tell what failed.*

```
2026-01-05T02:00:07Z ERROR job failed
$ grep -n "return err" jobs/sync.go | head -3
41:     return err
58:     return err
77:     return errors.New("failed")
```

The code returned bare errors or a constant message, so the log had no hint about which step or file failed. The team adopted a rule: wrap every returned error with what the function was doing and the key identifier, and log once at the top. The same failure now reads "sync users: fetch page 14: context deadline exceeded", which points directly at the slow call.

Common mistakes

- Ignoring errors with _
- Returning errors without context
- Comparing error text

MARGA by EknathaLabs marga.eknathalabs.com

- Deferring in a loop and holding files open

Check yourself

What does the %w verb in fmt.Errorf do?

Answer: Wraps an error so errors.Is and errors.As can find the cause

In what order do deferred calls run?

Answer: Last in, first out when the function returns

How should a program signal failure to the shell?

Answer: Print to stderr and exit with a non-zero status

When is panic appropriate?

Answer: For bugs and impossible states, not for expected failures

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Interfaces and packages

13 minutes. Use small interfaces to decouple code and make it testable, and organise a project into packages with clear boundaries.

A tool that calls the real cloud API on every test is slow, flaky and expensive. The usual remedy is to depend on a small behaviour, not on a concrete client, and to substitute a fake in tests. Go does this with interfaces, which are satisfied implicitly: a type implements an interface just by having the methods. Together with a sensible package layout, this keeps programs easy to test and change.

What you will learn

- Define and satisfy interfaces implicitly
- Accept interfaces, return concrete types
- Test with small fakes instead of real services
- Package layout, visibility and avoiding import cycles

Implicit interfaces

An interface is a set of method signatures. Any type with those methods satisfies it; there is no `implements` keyword.

```
type Notifier interface {
    Send(msg string) error
}

type Slack struct{ URL string }

func (s Slack) Send(msg string) error {
    // post to the webhook
    return nil
}

func alert(n Notifier, msg string) error {
    return n.Send("ALERT: " + msg)
}
```

`alert` works with Slack, email, a pager service or a test fake, without knowing which. Keep interfaces small, often one or two methods, such as `io.Reader` and `io.Writer` from the standard library. The wider an interface, the harder it is to fake and to satisfy.

Accept interfaces, return structs

A common guideline: functions **accept** the smallest interface they need and **return** concrete types. Define the interface where it is used (in the consumer package), not next to the implementation. That way the consumer owns the contract and the provider needs no knowledge of its callers. Do not create an interface for every struct "just in case"; add one when you have two implementations or when a test needs a fake.

MARGA by EknathaLabs marga.eknathalabs.com

Test with a fake

```
type fakeNotifier struct{ sent []string }

func (f *fakeNotifier) Send(msg string) error {
    f.sent = append(f.sent, msg)
    return nil
}

func TestAlertPrefix(t *testing.T) {
    f := &fakeNotifier{}
    if err := alert(f, "disk full"); err != nil {
        t.Fatal(err)
    }
    if got := f.sent[0]; got != "ALERT: disk full" {
        t.Errorf("got %q", got)
    }
}
```

The test needs no network and runs in microseconds. For richer behaviour, the fake can return a configured error to check how your code handles failure, which is hard to provoke against a real service. The empty interface (`any`) accepts everything and checks nothing at compile time; use it sparingly.

Packages

A package is a directory; all files in it share a name and a namespace. A typical layout for a tool:

```
diskcheck/
  go.mod
  main.go            package main: parses flags, wires things together
  internal/
    probe/           package probe: reads disk usage
    report/          package report: formats results
```

Names starting with a capital letter are exported. The `internal` directory can only be imported by code inside the parent module, which keeps private code private. Go forbids **import cycles**, so if two packages need each other, pull the shared piece into a third package or reverse a dependency with an interface. Keep `main` thin: it reads configuration and calls into packages that are easy to test.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Interfaces and packages fits together

Part	What it does	What goes wrong
Implicit interfaces	An interface is a set of method signatures.	Creating an interface for every type.
Accept interfaces, return structs	A common guideline: functions accept the smallest interface they need and return concrete types.	Making interfaces large.
Test with a fake	The test needs no network and runs in microseconds.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Packages	A package is a directory; all files in it share a name and a namespace.	Importing concrete clients deep inside business logic.

Flow: Implicit interfaces to Accept interfaces, return structs / Accept interfaces, return structs to Test with a fake / Test with a fake to Packages

Incident: The tests that called the real API

Incident: *A pipeline failed three times a week with timeouts, and no code had changed.*

```
--- FAIL: TestListInstances (30.00s)
    inventory_test.go:21: Get "https://ec2.eu-west-1.amazonaws.com/": context deadline exceeded
FAIL    example.com/ops/inventory   30.019s
```

The inventory package created a real cloud client inside its functions, so tests depended on credentials and network. The team introduced a one-method interface, Lister, passed it into the functions, and wrote a fake for tests. Unit tests now run in milliseconds and never touch the network; a small separate integration test, run nightly, checks the real API.

Common mistakes

- Creating an interface for every type
- Making interfaces large
- Importing concrete clients deep inside business logic
- Circular dependencies between packages

Check yourself

How does a Go type satisfy an interface?

Answer: By having all the methods, with no explicit declaration

Where should an interface usually be defined?

Answer: In the package that uses it

Why use a fake in unit tests?

Answer: To avoid the network and to provoke specific results and errors

What does the internal directory do?

Answer: Limits imports to code inside the parent module

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Goroutines, channels and sync

14 minutes. Run work concurrently with goroutines, coordinate with channels and WaitGroups, protect shared state, and find races early.

An operations tool often checks many things at once: a hundred hosts, a dozen endpoints, a queue of files. Done one by one, it takes minutes; done concurrently, seconds. Go makes concurrency cheap with goroutines, but concurrency also creates a new class of bugs: leaks, deadlocks and data races. A few patterns and one tool, the race detector, handle most of the risk.

What you will learn

- Start goroutines and wait for them with sync.WaitGroup
- Pass results with channels and know when to close them
- Protect shared data with sync.Mutex or avoid sharing
- Bound concurrency with a worker pool and detect races with -race

Goroutines and WaitGroup

A goroutine is a lightweight thread started with the `go` keyword. The program exits when `main` returns, even if goroutines are still running, so you must wait for them.

```
var wg sync.WaitGroup
for _, h := range hosts {
    wg.Add(1)
    go func(host string) {
        defer wg.Done()
        check(host)
    }(h)
}
wg.Wait()
```

Pass the loop variable as an argument (or rely on Go 1.22 and later, where each iteration has its own variable). In older code, capturing `h` directly meant that every goroutine saw the last host, a classic bug.

Channels

A channel carries values between goroutines and synchronises them.

```
results := make(chan Result)
for _, h := range hosts {
    go func() { results <- probe(h) }()
}
for range hosts {
    r := <-results
    fmt.Println(r.Host, r.OK)
}
```

An unbuffered channel blocks the sender until a receiver is ready; a buffered channel (`make(chan T, 10)`) blocks only when full. The **sender** closes a channel to say "no more values", and receivers can `range` over it until it closes. Sending on a closed channel panics, and a goroutine that blocks forever on a channel nobody reads is a **leak**, which shows up as slowly growing memory.

Shared state

If goroutines must share a variable, protect it:

```
type Counter struct {
    mu sync.Mutex
    n  map[string]int
}

func (c *Counter) Inc(k string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n[k]++
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

A plain map written by two goroutines is a **data race** and can crash with "concurrent map writes". Use a mutex, `sync/atomic` for simple counters, or better, give one goroutine ownership of the data and send it messages. The saying goes: share memory by communicating, not communicate by sharing memory.

Limit concurrency and test for races

Starting 50,000 goroutines that each open a connection will exhaust file descriptors or overload the target. Use a bounded worker pool:

```
jobs := make(chan string)
var wg sync.WaitGroup
for i := 0; i < 8; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for h := range jobs {
            check(h)
        }
    }()
}
for _, h := range hosts {
    jobs <- h
}
close(jobs)
wg.Wait()
```

Run tests with `go test -race ./...` and, in CI, build a test binary with the race detector. It reports unsynchronised access with stack traces of both goroutines. It slows the program, so use it in tests and staging, not in production.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Goroutines, channels and sync fits together

Part	What it does	What goes wrong
Goroutines and WaitGroup	A goroutine is a lightweight thread started with the go keyword.	Forgetting to wait for goroutines.
Channels	A channel carries values between goroutines and synchronises them.	Closing a channel from the receiving side.
Shared state	If goroutines must share a variable, protect it.	Unbounded goroutines.
Limit concurrency and test for races	Starting 50,000 goroutines that each open a connection will exhaust file descriptors or overload the target.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Goroutines and WaitGroup to Channels / Channels to Shared state / Shared state to Limit concurrency and test for races

Incident: Concurrent map writes in production

> **Incident:** *A health aggregator crashed every few days with a fatal error and no panic message to recover from.*

```
fatal error: concurrent map writes

goroutine 412 [running]:
runtime.throw({0x9f1a2c, 0x15})
main.(*Aggregator).record(...)
    /src/agg.go:57
```

Several probe goroutines wrote to the same map without a lock. The race detector found it immediately when the team ran the tests with -race, which they had never done. A mutex around the map fixed the crash, and CI now runs go test -race on every pull request. They also bounded the number of probes in flight.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Forgetting to wait for goroutines
- Closing a channel from the receiving side
- Unbounded goroutines
- Sharing maps without locks

Check yourself

What does sync.WaitGroup do?

Answer: Waits until a set of goroutines has finished

Who should close a channel?

Answer: The sender, when no more values will be sent

What is a data race?

Answer: Two goroutines accessing the same memory at once, with at least one writing, without synchronisation

Why use a worker pool?

Answer: To bound concurrency to what resources and targets can handle

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Context, timeouts and graceful shutdown

13 minutes. Pass context through your code so work can time out, be cancelled, and stop cleanly when the process is told to shut down.

The most common reliability bug in network code is a call that waits forever. A dependency slows down, every request piles up behind it, and the whole service stalls. Go's answer is the context package: a value passed down the call chain that carries a deadline, a cancellation signal and a few request-scoped values. Using it consistently is what separates a toy program from a service you can operate.

What you will learn

- Create contexts with timeouts and deadlines
- Pass ctx as the first argument and respect it
- Set client and server timeouts in net/http
- Shut down gracefully on SIGTERM

Contexts

A context is created from a parent and can be cancelled. Cancelling the parent cancels all children.

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
if err != nil {
    return err
}
resp, err := http.DefaultClient.Do(req)
if err != nil {
    return fmt.Errorf("get %s: %w", url, err)  // may be context deadline exceeded
}
defer resp.Body.Close()
```

Always call `cancel`, usually with `defer`, to release resources. The convention is that `ctx context.Context` is the first parameter of any function that does I/O or may block, and it is passed along, not stored in structs.

Respect cancellation in your own code

Loops and waits must check the context, or cancellation does nothing:

```
func poll(ctx context.Context, every time.Duration, f func() error) error {
    t := time.NewTicker(every)
    defer t.Stop()
    for {
        select {
        case <-ctx.Done():
            return ctx.Err()   // context.Canceled or DeadlineExceeded
        case <-t.C:
            if err := f(); err != nil {
                return err
            }
        }
    }
}
```

`select` waits on several channels and takes the first that is ready. A goroutine that never looks at `ctx.Done()` is the usual cause of leaks.

Timeouts in net/http

The default `http.Client` has **no timeout**, and a default server has none for slow clients. Set them explicitly:

```
client := &http.Client{Timeout: 10 * time.Second}
```

MARGA by EknathaLabs marga.eknathalabs.com

```
srv := &http.Server{
    Addr:              ":8080",
    Handler:           mux,
    ReadHeaderTimeout: 5 * time.Second,
    ReadTimeout:       15 * time.Second,
    WriteTimeout:      30 * time.Second,
    IdleTimeout:       60 * time.Second,
}
```

Choose timeouts from what your callers can tolerate, and keep inner timeouts shorter than outer ones so the outer caller still has time to retry or fail cleanly.

Graceful shutdown

When Kubernetes or systemd stops your service, it sends SIGTERM. A graceful service stops accepting new requests, finishes the ones in flight, then exits.

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

go func() { _ = srv.ListenAndServe() }()

<-ctx.Done()
shutdownCtx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
    log.Printf("forced shutdown: %v", err)
}
```

Make the shutdown timeout shorter than the orchestrator's grace period (30 seconds by default in Kubernetes), or the process will be killed in the middle of the work.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Context, timeouts and graceful shutdown fits together

Part	What it does	What goes wrong
Contexts	A context is created from a parent and can be cancelled.	Using http.DefaultClient with no timeout.
Respect cancellation in your own	Loops and waits must check the context, or cancellation does nothing.	Never calling cancel.
Timeouts in net/http	The default http.Client has no timeout, and a default server has none for slow clients.	Shutdown timeouts longer than the grace period.
Graceful shutdown	When Kubernetes or systemd stops your service, it sends SIGTERM.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Contexts to Respect cancellation in your own / Respect cancellation in your own to Timeouts in net/http / Timeouts in net/http to Graceful shutdown

Incident: The service that never finished a deploy

Incident: *Each rollout took ten minutes and ended with dropped requests.*

```
$ kubectl rollout status deploy/payments
Waiting for deployment "payments" rollout to finish: 1 old replicas are pending termination...
$ kubectl logs payments-7c9 --previous | tail -2
2026-01-05T11:02:41Z listening on :8080
```

The program ignored SIGTERM, so Kubernetes waited the full 30-second grace period and then killed it, dropping open requests. The team added signal.NotifyContext and srv.Shutdown with a 20-second limit, plus a readiness check that turns false at shutdown start. Rollouts now finish in a minute with no errors.

Common mistakes

- Using http.DefaultClient with no timeout
- Storing contexts in structs
- Never calling cancel
- Shutdown timeouts longer than the grace period

Check yourself

What does context.WithTimeout return?

Answer: A derived context and a cancel function

Where does ctx go in a function signature?

Answer: As the first parameter

What is the default timeout of http.Client?

Answer: None, requests can wait forever

What should a service do on SIGTERM?

Answer: Stop accepting new work, finish in-flight work, then exit

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. HTTP services, clients and JSON

13 minutes. Build a small HTTP service with handlers, middleware, health endpoints and JSON, and call other services safely.

Much operations work is an API: a webhook receiver, a metrics exporter, a small internal service that Kubernetes probes and other teams call. The Go standard library is enough to build these without a framework. This lesson shows the shape of a clean service: routing, JSON in and out, a health endpoint, a bit of middleware, and a client that behaves well.

What you will learn

- Write handlers and route requests with http.ServeMux
- Encode and decode JSON with struct tags and validation
- Add middleware for logging and recovery
- Expose health endpoints and call other services with a tuned client

Handlers and routing

```
mux := http.NewServeMux()
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    fmt.Fprintln(w, "ok")
})
mux.HandleFunc("GET /hosts/{name}", getHost)
mux.HandleFunc("POST /hosts", createHost)
```

Since Go 1.22 the standard router understands methods and path variables like `{name}`, read with `r.PathValue("name")`. A handler receives the response writer and the request, and you set headers and status before writing the body. Return an appropriate code: 400 for bad input, 404 for missing, 500 for your own bugs.

JSON in and out

```
type Host struct {
    Name string `json:"name"`
    Port int    `json:"port"`
}

func createHost(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20)
    var h Host
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(&h); err != nil {
        http.Error(w, "bad json: "+err.Error(), http.StatusBadRequest)
        return
    }
    if h.Name == "" || h.Port < 1 || h.Port > 65535 {
        http.Error(w, "name and a valid port are required", http.StatusBadRequest)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    _ = json.NewEncoder(w).Encode(h)
}
```

Limit the body size, reject unknown fields if your API is strict, and validate the values, because decoding only checks the shape. Struct tags name the fields; `omitempty` skips empty ones.

Middleware

Middleware wraps a handler to add behaviour such as logging or panic recovery.

```
func logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
```

MARGA by EknathaLabs marga.eknathalabs.com

```
        next.ServeHTTP(w, r)
        log.Printf("%s %s %s", r.Method, r.URL.Path, time.Since(start))
    })
}

srv := &http.Server{Addr: ":8080", Handler: logging(mux)}
```

Chain them for recovery, request IDs, authentication and metrics. Use structured logging (`log/slog`) so fields such as the request ID can be searched.

Health endpoints and clients

Expose two endpoints: **liveness** (`/healthz`, the process is not stuck) and **readiness** (`/readyz`, it can take traffic: database reachable, caches warm). Kubernetes probes use them. Keep liveness cheap and do not make it depend on the database, or a database blip will restart every pod.

For outbound calls, reuse one `http.Client` (it pools connections), set `Timeout`, always close `resp.Body`, check the status code, and retry only idempotent requests with backoff and a limit. Never build a new client per request; it leaks connections and sockets.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How HTTP services, clients and JSON fits together

Part	What it does	What goes wrong
Handlers and routing	Since Go 1.22 the standard router understands methods and path variables like {name}, read with r.PathValue("name").	No body size limit on requests.
JSON in and out	Limit the body size, reject unknown fields if your API is strict, and validate the values, because decoding only checks the shape.	Skipping validation after decoding.
Middleware	Middleware wraps a handler to add behaviour such as logging or panic recovery.	Creating a client per request.
Health endpoints and clients	Expose two endpoints: liveness (/healthz, the process is not stuck) and readiness (/readyz, it can take traffic: database reachable, caches warm).	Liveness probes that check dependencies.

Flow: Handlers and routing to JSON in and out / JSON in and out to Middleware / Middleware to Health endpoints and clients

Incident: A fresh client for each request

Incident: *A collector service slowly ran out of sockets and began failing with connection errors.*

```
dial tcp 10.2.4.7:443: connect: cannot assign requested address
$ ss -tan | grep -c TIME-WAIT
61204
$ grep -n "http.Client{" collector/*.go
collector/fetch.go:31:  c := &http.Client{}
```

Each fetch created a new client with its own transport, so connections were never reused and thousands sat in TIME-WAIT. The team created one shared client with a timeout and a tuned transport, closed response bodies in all code paths, and added a metric for open connections. Socket use dropped by 99 percent.

Common mistakes

- No body size limit on requests
- Skipping validation after decoding
- Creating a client per request

- Liveness probes that check dependencies

Check yourself

What does a JSON struct tag do?

Answer: Controls the field name used when encoding and decoding

Why limit the request body?

Answer: To protect memory from huge or malicious requests

What is readiness for?

Answer: Telling the orchestrator whether the service can take traffic now

How should you use http.Client?

Answer: Create one, reuse it, and set a timeout

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Testing, benchmarks and tooling

12 minutes. Write table-driven tests, measure coverage, benchmark hot paths and use the toolchain checks in CI.

Go has testing built in: a file ending in _test.go, a function starting with Test, and the go test command. There is no framework to choose or install. Combined with the race detector, benchmarks and vet, you get a solid safety net with very little setup, which is why Go projects are easy to contribute to and safe to change.

What you will learn

- Write table-driven unit tests with subtests
- Use test helpers, temporary directories and fakes
- Measure coverage and write benchmarks
- Run vet, race and linters in CI

A first test

Tests live next to the code in `_test.go` files.

```
// parse.go
func ParseSize(s string) (int64, error) { /* ... */ }

// parse_test.go
func TestParseSize(t *testing.T) {
    tests := []struct {
        name    string
        in      string
        want    int64
        wantErr bool
    }{
        {"bytes", "512", 512, false},
        {"kilobytes", "2K", 2048, false},
        {"empty", "", 0, true},
        {"junk", "12Q", 0, true},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            got, err := ParseSize(tc.in)
            if (err != nil) != tc.wantErr {
                t.Fatalf("err = %v, wantErr %v", err, tc.wantErr)
            }
            if got != tc.want {
                t.Errorf("got %d, want %d", got, tc.want)
            }
        })
    }
}
```

The **table-driven** style makes adding a case one line and gives each case a name in the output.

Helpers and temporary files

`t.TempDir()` gives a directory that is cleaned up after the test. `t.Helper()` marks a function as a helper so failures point at the caller. `t.Cleanup(fn)` registers cleanup. Use `httptest.NewServer` to start a fake HTTP server for client code, and `httptest.NewRecorder` to test handlers without a network. Combine with the fakes from the interfaces lesson. Keep tests deterministic: inject the clock and random sources instead of calling `time.Now` deep inside logic.

Coverage and benchmarks

```
go test ./... -cover
go test ./... -coverprofile=cover.out
go tool cover -html=cover.out
```

MARGA by EknathaLabs marga.eknathalabs.com

Coverage shows what was executed, not whether it was checked well; use it to find untested branches, not as a target for its own sake. Benchmarks measure speed:

```
func BenchmarkParseSize(b *testing.B) {
    for i := 0; i < b.N; i++ {
        _, _ = ParseSize("512M")
    }
}

go test -bench=. -benchmem
```

Read `ns/op` and `allocs/op`, and compare before and after a change; the `benchstat` tool helps with noise. Measure first, then optimise.

Tooling in CI

A reasonable pipeline for a Go service:

```
gofmt -l . | tee /dev/stderr | (! read)   # fail if anything is unformatted
go vet ./...
go test -race -cover ./...
go build ./...
govulncheck ./...                         # known vulnerabilities in dependencies and code
```

Add a linter such as golangci-lint if the team wants more rules. Cache the module and build directories between runs to keep the pipeline fast. Pin the Go version in `go.mod` so developers and CI use the same toolchain.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Testing, benchmarks and tooling fits together

Part	What it does	What goes wrong
A first test	Tests live next to the code in _test.go files.	Tests that depend on the network or the clock.
Helpers and temporary files	t.TempDir() gives a directory that is cleaned up after the test.	Not using subtests, so failures lack names.
Coverage and benchmarks	Coverage shows what was executed, not whether it was checked well; use it to find untested branches, not as a target for its own sake.	Chasing coverage numbers.
Tooling in CI	A reasonable pipeline for a Go service.	Skipping the race detector.

Flow: A first test to Helpers and temporary files / Helpers and temporary files to Coverage and benchmarks / Coverage and benchmarks to Tooling in CI

Incident: The 100 percent coverage that missed the bug

Incident: *A report parser had full coverage and still sent wrong numbers to finance.*

```
$ go test ./report -cover
ok      example.com/ops/report  0.012s  coverage: 100.0% of statements
$ go test ./report -run TestMonth -v
--- PASS: TestMonth (0.00s)
$ # month 12 is parsed as 0
```

Tests executed every line but only with a single month value that happened to work. The team replaced the example test with a table covering every month and boundary values, and added a rule that new parsing code needs edge cases (empty, minimum, maximum, invalid). They stopped using the coverage percentage as a goal and use it to find unexecuted branches.

Common mistakes

- Tests that depend on the network or the clock

MARGA by EknathaLabs marga.eknathalabs.com

- Chasing coverage numbers
- Not using subtests, so failures lack names
- Skipping the race detector

Check yourself

What is a table-driven test?

Answer: A list of cases run in a loop with subtests

What does t.TempDir() provide?

Answer: A temporary directory removed after the test

What does go test -bench=. measure?

Answer: Speed and allocations of benchmark functions

Which command finds known vulnerabilities in dependencies?

Answer: govulncheck

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Building CLI tools and small container images

13 minutes. Ship Go programs well: a clear command-line interface, version information, multi-stage builds and tiny, safe container images.

You wrote the tool, and now other people must run it: on laptops, in pipelines and in containers. A clear command-line interface, a version flag that tells you which build is running, and a small image with no shell for an attacker are the finishing touches that make a Go tool operable. Go makes each of these unusually easy.

What you will learn

- Parse flags and subcommands, and set exit codes
- Embed the version at build time with ldflags
- Write a multi-stage Dockerfile for a static binary
- Choose a base image: scratch, distroless or alpine

A command-line tool

The standard `flag` package is enough for simple tools:

```
func main() {
    addr := flag.String("addr", ":8080", "listen address")
    verbose := flag.Bool("v", false, "verbose logging")
    showVersion := flag.Bool("version", false, "print version and exit")
    flag.Parse()

    if *showVersion {
        fmt.Println(version)
        return
    }
    // run the tool with *addr and *verbose
}
```

For tools with many subcommands (`tool get`, `tool apply`), libraries such as cobra are common. Whatever you use: print usage on `-h`, write results to standard output and diagnostics to standard error, and exit non-zero on failure so pipelines can react. Read secrets from environment variables or files, not from flags, because flags show in the process list.

Embed the version

Put a variable in `main` and set it at build time:

```
var version = "dev"
```

```
go build -ldflags "-s -w -X main.version=$(git describe --tags --always)" -o tool .
./tool -version
```

`-X` sets the string, and `-s -w` strips debug data to make the binary smaller. Every binary and container should report exactly what it is, which makes incident questions like "which build is running?" easy to answer. Go also records module and VCS information in the binary, readable with `go version -m ./tool`.

A multi-stage image

Compile in a full Go image and copy only the binary into a minimal one:

```
FROM golang:1.24 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags "-s -w" -o /out/tool .

FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/tool /tool
```

MARGA by EknathaLabs marga.eknathalabs.com

```
USER nonroot
ENTRYPOINT ["/tool"]
```

Copying `go.mod` and `go.sum` first lets the dependency download layer be cached when only code changes. The final image holds one static binary and nothing else, often under 15 MB, with no shell, no package manager and few vulnerabilities to scan. The image runs as a non-root user.

Choosing the base

`scratch` is an empty image: smallest, but no CA certificates, time zone data or user accounts unless you copy them in. **Distroless** static images include certificates and a non-root user and nothing else, a good default. **Alpine** gives you a shell and package manager for debugging, at the cost of more surface and a different C library (not an issue for static Go builds). For debugging a distroless image in Kubernetes, use an ephemeral debug container instead of installing tools in production. Add a health check or rely on the orchestrator probes, and set resource limits, including `GOMEMLIMIT` so the Go runtime respects the container memory limit.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Building CLI tools and small container images fits together

Part	What it does	What goes wrong
A command-line tool	The standard flag package is enough for simple tools.	Putting secrets in command-line flags.
Embed the version	Put a variable in main and set it at build time.	Running as root in the container.
A multi-stage image	Compile in a full Go image and copy only the binary into a minimal one.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Choosing the base	scratch is an empty image: smallest, but no CA certificates, time zone data or user accounts unless you copy them in.	Forgetting CA certificates in scratch images.

Flow: A command-line tool to Embed the version / Embed the version to A multi-stage image / A multi-stage image to Choosing the base

Incident: Which build is running?

Incident: *A bug fix did not seem to work, and nobody could say whether production had the new binary.*

```
$ kubectl exec deploy/billing -- /billing -version
OCI runtime exec failed: exec: "/billing": stat /billing: no such file or directory
$ kubectl get deploy billing -o jsonpath="{.spec.template.spec.containers[0].image}"
registry.example.com/billing:latest
```

The image used the latest tag and the binary had no version output, so nobody could confirm what ran. The team started tagging images with the commit hash, embedding the version with ldflags, exposing it at /version and in a metric, and recorded it in release notes. A debugging container showed the build info in seconds.

Common mistakes

- Using latest tags
- Running as root in the container
- Putting secrets in command-line flags
- Forgetting CA certificates in scratch images

Check yourself

How do you embed a version string at build time?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: With -ldflags "-X main.version=..."

Why use a multi-stage build?

Answer: To ship only the binary in a small final image

What does CGO_ENABLED=0 give you?

Answer: A static binary with no dependency on the C library

Where should secrets be passed to a CLI tool?

Answer: Through environment variables or files, not command-line flags

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Debugging Go programs: panics, leaks, races and profiles

14 minutes. Read a Go stack trace, find goroutine leaks and races, and use pprof to see where time and memory go.

A Go service in production fails in a few characteristic ways: a panic with a stack trace, memory that grows until the process is killed, CPU that spikes, or requests that hang. Go ships with excellent diagnostic tools, and most of them work on a running process. Knowing the handful of commands turns a vague report into a specific line of code.

What you will learn

- Read a panic and a goroutine dump
- Find goroutine leaks and deadlocks
- Profile CPU and memory with pprof
- Use the race detector and delve for deeper investigation

Read the trace

```
panic: runtime error: invalid memory address or nil pointer dereference
[signal SIGSEGV: segmentation violation code=0x1 addr=0x0 pc=0x4a1f3c]

goroutine 33 [running]:
main.(*Client).Fetch(0x0, {0xc000014080, 0x9})
    /src/client.go:48 +0x1c
main.handle(...)
    /src/server.go:91
```

Read from the top: the first line says what happened, the first frame in your own code shows where (`client.go:48`), and the receiver value `0x0` shows the method was called on a nil pointer. Frames below show who called it. Most panics are nil pointers, out-of-range indexes, closed channels and nil map writes, each visible in that first line.

Leaks and hangs

If requests hang or memory grows, look at goroutines. Send `SIGQUIT` (Ctrl+backslash) to dump all stacks, or expose pprof:

```
import _ "net/http/pprof"

go func() { log.Println(http.ListenAndServe("127.0.0.1:6060", nil)) }()

curl -s "http://127.0.0.1:6060/debug/pprof/goroutine?debug=1" | head -30
```

Thousands of goroutines at the same line, for example `chan send` or `select`, is a leak: something is waiting for a receiver or a cancellation that never comes. `all goroutines are asleep - deadlock!` means every goroutine is blocked, usually on a channel or a lock. Bind the pprof port to localhost and reach it through a port-forward, never expose it publicly.

Profiling

```
go tool pprof -http=:0 "http://127.0.0.1:6060/debug/pprof/profile?seconds=30"   # CPU
go tool pprof -http=:0 "http://127.0.0.1:6060/debug/pprof/heap"                 # memory in use
```

The flame graph and `top` view show which functions use the most CPU or hold the most memory. For memory, compare two heap profiles taken some minutes apart (`-base`) to see what grows. Also check `GODEBUG=gctrace=1` for garbage-collection pressure, and the `runtime/metrics` or Prometheus Go collector for goroutine count, heap size and GC pauses on dashboards. Profile before guessing: the slow part is rarely where you expect.

MARGA by EknathaLabs
marga.eknathalabs.com

Races and the debugger

The race detector (`go test -race`, or `go build -race` for a staging binary) reports two goroutines touching the same variable. Treat each report as a real bug. For logic errors, **delve** (`dlv debug`, `dlv test`, `dlv attach PID`) lets you set breakpoints and inspect variables, including goroutines. Add structured logging with request IDs, and in incidents capture a goroutine dump and a heap profile before restarting, because a restart destroys the evidence.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Debugging Go programs: panics, leaks, races and... fits together

Part	What it does	What goes wrong
Read the trace	Read from the top: the first line says what happened, the first frame in your own code shows where (client.go:48), and the receiver value 0x0 shows the method was called...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Leaks and hangs	If requests hang or memory grows, look at goroutines.	Exposing pprof publicly.
Profiling	The flame graph and top view show which functions use the most CPU or hold the most memory.	Guessing at performance without a profile.
Races and the debugger	The race detector (go test -race, or go build -race for a staging binary) reports two goroutines touching the same variable.	Restarting before capturing a dump.

Flow: Read the trace to Leaks and hangs / Leaks and hangs to Profiling / Profiling to Races and the debugger

Incident: Forty thousand goroutines

Incident: *Memory in a webhook forwarder climbed 100 MB per hour until the pod was killed.*

```
$ curl -s "http://127.0.0.1:6060/debug/pprof/goroutine?debug=1" | head -4
goroutine profile: total 40871
40112 @ 0x43a2e6 0x4062b9 0x4057d5
#   0x4057d4    main.forward+0x94   /src/forward.go:63
```

Each webhook started a goroutine that sent to an unbuffered results channel, but the receiver returned early on timeout, leaving the sender blocked forever. The fix used a buffered channel of size one and checked ctx.Done() in the sender. A goroutine-count alert was added to the dashboard so a repeat would be seen within an hour.

Common mistakes

- Restarting before capturing a dump
- Exposing pprof publicly
- Guessing at performance without a profile
- Ignoring race detector reports

Check yourself

What does the first frame in your own code in a panic trace show?

Answer: Where the failing code is

A goroutine count that keeps rising points to what?

Answer: A goroutine leak, often blocked on a channel or missing cancellation

How do you find what is using CPU?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Take a CPU profile with pprof and view the flame graph

How should the pprof port be exposed?

Answer: Bind to localhost and reach it by port-forward

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Configuration, logging and metrics in Go services

12 minutes. Read configuration from flags and environment, log with slog and expose Prometheus metrics so Go services are easy to run and observe.

The code that runs your logic is only part of a production service. It must also accept configuration safely, explain itself in logs and expose numbers that tools can graph.

What you will learn

- Configuration with flags, environment and validation
- Structured logging with log/slog
- Metrics with the Prometheus client
- Health endpoints and redaction

Configuration

```
package main

import (
    "flag"
    "fmt"
    "os"
    "time"
)

type Config struct {
    Addr    string
    DSN     string
    Timeout time.Duration
}

func load() (Config, error) {
    c := Config{}
    flag.StringVar(&c.Addr, "addr", envOr("ADDR", ":8080"), "listen address")
    flag.DurationVar(&c.Timeout, "timeout", 5*time.Second, "request timeout")
    flag.Parse()
    c.DSN = os.Getenv("DATABASE_DSN")
    if c.DSN == "" {
        return c, fmt.Errorf("DATABASE_DSN is required")
    }
    return c, nil
}

func envOr(k, d string) string {
   if v := os.Getenv(k); v != "" {
       return v
    }
    return d
}
```

Validate at start and fail fast with a clear message. Never log the DSN or any secret.

Logging with slog

```
logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{Level: slog.LevelInfo}))
logger.Info("request", "request_id", id, "method", r.Method, "path", r.URL.Path, "status", status, \
     "duration_ms", ms)
logger.Error("db query failed", "err", err, "request_id", id)
```

JSON logs with stable keys can be searched and aggregated. Include a request ID in every line for a request and pass it through the context.

MARGA by EknathaLabs marga.eknathalabs.com

Metrics and health

```
var reqs = promauto.NewCounterVec(
    prometheus.CounterOpts{Name: "http_requests_total", Help: "Requests"},
    []string{"method", "code"},
)
var dur = promauto.NewHistogram(prometheus.HistogramOpts{Name: "http_request_duration_seconds", Bu \
    ckets: prometheus.DefBuckets})

http.Handle("/metrics", promhttp.Handler())
http.HandleFunc("/healthz", func(w http.ResponseWriter, _ *http.Request) { w.WriteHeader(200) })
```

Keep metric labels low-cardinality: never use user IDs or full URLs. Separate **liveness** from **readiness** checks, and make readiness reflect dependencies such as the database.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Configuration, logging and metrics in Go services fits together

Part	What it does	What goes wrong
Configuration	Validate at start and fail fast with a clear message.	Free-text logs with no identifiers.
Logging with slog	JSON logs with stable keys can be searched and aggregated.	Logging secrets or full connection strings.
Metrics and health	Keep metric labels low-cardinality: never use user IDs or full URLs.	High-cardinality metric labels.

Flow: Configuration to Logging with slog / Logging with slog to Metrics and health

Incident: One request in a haystack

Incident: *Free-text logs made tracing a single failing request very slow.*

```
14:07:11 ERROR something went wrong
14:07:11 INFO handled order
14:07:12 ERROR something went wrong
customer reported failure at 14:07, no request id to search for
```

The logs had no stable fields or identifiers, so engineers had to read thousands of lines. The team moved to structured logs with request IDs, returned the ID to clients in a header and made support quote it.

Common mistakes

- Free-text logs with no identifiers
- High-cardinality metric labels
- Logging secrets or full connection strings
- One health check for liveness and readiness

Check yourself

What does log/slog give you?

Answer: Structured, leveled logging

What should every request log line include?

Answer: A request ID

What is wrong with a user ID label on metrics?

Answer: Too many unique series

What should readiness reflect?

Answer: Whether dependencies are usable

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Writing Kubernetes clients and controllers in Go

13 minutes. Use client-go to list and watch resources, understand informers and work queues, and write a small, safe reconcile loop.

Many Kubernetes extensions are small Go programs following one idea: watch objects, compare desired and actual state, and make a change. Understanding the pieces helps you write them and read the ones you operate.

What you will learn

- A client with client-go and in-cluster configuration
- Informers, caches and work queues
- A reconcile loop that is idempotent
- Operating controllers: leader election, RBAC and metrics

A simple client

```
cfg, err := rest.InClusterConfig()
if err != nil {
    cfg, err = clientcmd.BuildConfigFromFlags("", os.Getenv("KUBECONFIG"))
}
if err != nil {
    log.Fatal(err)
}
cs := kubernetes.NewForConfigOrDie(cfg)

pods, err := cs.CoreV1().Pods("shop").List(ctx, metav1.ListOptions{LabelSelector: "app=web"})
```

In a cluster the program uses the service account token automatically, so RBAC must allow exactly the verbs it needs.

Informers and queues

```
factory := informers.NewSharedInformerFactoryWithOptions(cs, 10*time.Minute, informers.WithNamespa \
    ce("shop"))
podInformer := factory.Core().V1().Pods()
queue := workqueue.NewTypedRateLimitingQueue(workqueue.DefaultTypedControllerRateLimiter[string]() \
    )

podInformer.Informer().AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        if key, err := cache.MetaNamespaceKeyFunc(obj); err == nil {
            queue.Add(key)
        }
    },
})
factory.Start(stopCh)
cache.WaitForCacheSync(stopCh, podInformer.Informer().HasSynced)
```

The **informer** keeps a local cache updated by one watch, so reads do not hit the API server. Events add object **keys** to a **work queue**, which removes duplicates and retries with backoff.

Reconcile

```
func (c *Controller) reconcile(ctx context.Context, key string) error {
    ns, name, _ := cache.SplitMetaNamespaceKey(key)
    pod, err := c.podLister.Pods(ns).Get(name)
    if apierrors.IsNotFound(err) {
        return nil // already gone, nothing to do
    }
    if err != nil {
        return err
    }
    return c.ensureDesiredState(ctx, pod) // compare and fix; safe to run again
}
```

MARGA by EknathaLabs
marga.eknathalabs.com

The function looks at the **current** state and makes it match the desired state. It must be safe to run repeatedly and in any order.

Operate

Use **controller-runtime** or Kubebuilder for custom resources. Run two replicas with **leader election**, give the service account minimal RBAC, expose metrics for queue depth and reconcile errors, and set resource limits.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Writing Kubernetes clients and controllers in Go fits together

Part	What it does	What goes wrong
A simple client	In a cluster the program uses the service account token automatically, so RBAC must allow exactly the verbs it needs.	Broad RBAC for controllers.
Informers and queues	The informer keeps a local cache updated by one watch, so reads do not hit the API server.	Informers and queues The informer keeps a local cache updated by one watch, so reads do not hit the API server.
Reconcile	The function looks at the current state and makes it match the desired state.	Reconcile functions that are not idempotent.
Operate	Use controller-runtime or Kubebuilder for custom resources.	Running a single controller replica without leader election.

Flow: A simple client to Informers and queues / Informers and queues to Reconcile / Reconcile to Operate

Incident: Polling the API every second

Incident: *A script listing all pods in a loop slowed the control plane.*

```
verb=LIST resource=pods user-agent=pod-watcher-script
60 requests per minute from one client
API p99 latency increased 4x
```

The script listed all pods every second across many clusters. Replacing it with an informer reduced requests to a single watch and made the program react faster. The team also added a review rule against polling in controllers.

Common mistakes

- Polling instead of watching
- Reconcile functions that are not idempotent
- Running a single controller replica without leader election
- Broad RBAC for controllers

Check yourself

What does an informer provide?

Answer: A local cache fed by a watch

What does a work queue do?

Answer: Deduplicates and retries keys

What should reconcile do?

Answer: Make actual state match desired state

Why use leader election?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Only one active controller makes changes

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Go

Toolchain commands, types, errors, interfaces, goroutines, context, HTTP, testing and build flags for services and CLIs.

Toolchain

```
go version
go mod init example.com/shop
go mod tidy
go build ./...
go run ./cmd/api
go test ./... -race -cover
go vet ./...
gofmt -l .
go install ./cmd/api
```

> **Warning:** *Commit* `go.mod` *and* `go.sum`*. Without* `go.sum`*, builds cannot verify that dependencies are unchanged.*

Types, slices and maps

```
type Order struct {
    ID    string
    Items []Item
    Tags  map[string]string
}

names := make([]string, 0, 8)
names = append(names, "a")
m := map[string]int{"a": 1}
if v, ok := m["b"]; !ok { _ = v }
```

> **Warning:** *Writing to a nil map panics. Create maps with* `make` *or a literal. Reading from a nil map is safe.*

Errors and defer

```
f, err := os.Open(path)
if err != nil {
    return fmt.Errorf("open config %q: %w", path, err)
}
defer f.Close()

if errors.Is(err, os.ErrNotExist) { /* handle */ }
var pe *fs.PathError
if errors.As(err, &pe) { /* inspect */ }
```

> **Warning:** `defer` *in a loop runs when the function returns, not each iteration. Move the body into a function to release resources early.*

Interfaces

```
type Store interface {
    Get(ctx context.Context, id string) (Order, error)
}

func NewService(s Store) *Service { return &Service{store: s} }
```

Accept interfaces, return concrete types. Keep interfaces small and define them where they are used.

Goroutines and channels

```
var wg sync.WaitGroup
results := make(chan int, len(jobs))
for _, j := range jobs {
    wg.Add(1)
    go func() {
        defer wg.Done()
        results <- work(j)
    }()
}
wg.Wait()
close(results)
```

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *A goroutine that blocks forever on a channel is a leak. Always have a way to stop it: close the channel or cancel a context.*

Context

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()
req, _ := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
select {
case <-ctx.Done():
    return ctx.Err()
case r := <-ch:
    _ = r
}
```

Pass `ctx` as the first parameter. Never store it in a struct.

HTTP service

```
mux := http.NewServeMux()
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
})
srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 * time.Second}
go srv.ListenAndServe()
// on shutdown signal:
srv.Shutdown(ctx)
```

Warning: `http.ListenAndServe` *and the default client have no timeouts. Set them explicitly, or slow peers can hold connections forever.*

Testing

```
func TestSum(t *testing.T) {
    cases := []struct{ name string; a, b, want int }{
        {"zero", 0, 0, 0}, {"pos", 1, 2, 3},
    }
    for _, c := range cases {
        t.Run(c.name, func(t *testing.T) {
            if got := Sum(c.a, c.b); got != c.want { t.Fatalf("got %d want %d", got, c.want) }
        })
    }
}
```

```
go test -run TestSum -v ./pkg/...
go test -bench . -benchmem
go test -race ./...
```

Build for containers

```
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -trimpath -ldflags="-s -w" -o app ./cmd/api
```

```
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app ./cmd/api

FROM gcr.io/distroless/static-debian12
COPY --from=build /app /app
USER 65532:65532
ENTRYPOINT ["/app"]
```

Warning: *Pin the Go version in* `go.mod` *and in the build image, so local and CI builds use the same toolchain.*

Debugging

```
go test -race ./...
go tool pprof -http=:0 http://localhost:6060/debug/pprof/profile
curl -s localhost:6060/debug/pprof/goroutine?debug=1 | head -30
GODEBUG=gctrace=1 ./app
```

MARGA by EknathaLabs
marga.eknathalabs.com

Warning: *Never expose the pprof endpoint on a public port. Bind it to localhost or an internal network.*

MARGA by EknathaLabs
marga.eknathalabs.com